

Parallel Subsequence Matching in Time Series: A Comparative Performance Study of OpenMP (AoS vs SoA) and CUDA

Jacopo Bruscatelli

Department of Information Engineering (DINFO)

University of Florence, Italy

`jacopo.bruscatelli@stud.unifi.it`

September 17, 2026

Future Distribution Permission: The author gives permission for this document to be distributed to Unifi-affiliated students taking future courses.

Abstract

Subsequence similarity matching across large-scale time series datasets represents an embarrassingly parallel, compute- and memory-bound problem of fundamental importance in signal processing and anomaly detection. In this project, we implement and benchmark three algorithmic paradigms for multi-query matching using the Sum of Absolute Differences (SAD) metric evaluated on the *FaultDetectionA* dataset. We systematically analyze memory access patterns by contrasting Array of Structures (AoS) against Structure of Arrays (SoA) layouts under CPU multicore execution via OpenMP, and we assess massive throughput acceleration using an optimized CUDA kernel offloaded to an NVIDIA Tesla T4 GPU. Following a rigorous experimental benchmarking methodology featuring dedicated warm-up cycles, repeated multi-run executions, and statistical dispersion sampling, we evaluate performance in terms of execution time, speedup, and scaling behavior across varying query lengths and batch sizes.

1 Introduction

Time series subsequence matching is a foundational primitive applied across diverse real-world domains, including Industrial Fault Detection, medical monitoring (e.g., ECG anomaly segmentation), and financial forecasting. Given a repository of time series and an arbitrary query, the notation is defined as follows:

- M : The **number of univariate time series** stored in the dataset.
- N : The **length of each time series** (total number of discrete observations/samples per series).

- Q : The **query subsequence** (the target pattern to locate within the dataset).
- L : The **length of the query** Q , subject to the constraint $L \leq N$.
- i : The **starting offset** (candidate alignment index) of the sliding window within a series, where $0 \leq i \leq N - L$.
- i^* : The **optimal alignment offset** that minimizes the distance metric, identifying the best matching subsequence.
- W : The **total number of candidate sliding windows** evaluated per series, given by $W = N - L + 1$ (or $W = M \cdot (N - L + 1)$ across the entire dataset).

From a computational complexity point, the pairwise sliding-window exploration requires evaluating:

$$W = M \cdot (N - L + 1) \quad (1)$$

subsequence number of comparisons for every single query. When processing batches of K queries simultaneously, the total number of floating-point distance checks grows as $\mathcal{O}(K \cdot M \cdot N \cdot L)$.

Because individual queries and series comparisons are fully independent of one another, the task exhibits an *embarrassingly parallel* structure. Nevertheless, attaining high computational throughput on modern memory-hierarchical processors is strictly constrained by spatial locality, memory bandwidth, SIMD vectorization, and thread-scheduling overheads. This work presents:

1. A baseline single-threaded implementation in C++.
2. A parallel multicore CPU implementation utilizing OpenMP work-sharing directives, evaluating both Array of Structures (AoS) and Structure of Arrays (SoA) memory organizations.
3. A massively parallel GPU offloading strategy using NVIDIA CUDA designed to maximize global memory coalescing on device memory.

2 Methodology and Memory Architectures

2.1 Mathematical Formulation

The similarity between a query vector $Q = [q_0, q_1, \dots, q_{L-1}]$ and a sequence window $S[i \dots i + L - 1]$ is computed using the Sum of Absolute Differences (SAD) metric:

$$\mathcal{D}(Q, S, i) = \sum_{j=0}^{L-1} |S[i + j] - q_j| \quad (2)$$

The optimal match represents the index i^* that minimizes the distance functional:

$$i^* = \arg \min_{0 \leq i \leq N-L} \mathcal{D}(Q, S, i) \quad (3)$$

2.2 Memory Layout: Array of Structures (AoS) vs. Structure of Arrays (SoA)

Data layout in physical RAM governs cache line hit rates, prefetching behavior, and coalescing capabilities:

Array of Structures (AoS): Each time series is modeled as an independent object instance storing its identification metadata along with a heap-allocated dynamic array:

```
struct TimeSeries {
    int id;
    std::vector<double> data; // Dynamic allocation per instance
};
std::vector<TimeSeries> datasetAoS;
```

Traversing instances introduces "pointer chasing", scattered heap allocations, and suboptimal CPU cache line utilization due to metadata overhead.

Structure of Arrays (SoA): All observations across all sequences are flattened into a single, contiguous linear buffer of double-precision floating-point numbers:

```
struct TimeSeries_SoA {
    std::vector<double> all_data_flat; // Total size: M * N elements
    int serie_length;                // N
    std::vector<int> all_classes;     // Contiguous metadata
};
```

By strictly linearizing the dataset, subsequent reads follow regular strides, which maximizes hardware prefetch efficiency on host CPUs and enables 128-byte coalesced memory transactions across active warps on NVIDIA GPUs.

3 Parallel Implementations

3.1 OpenMP Multithreading (CPU)

The multicore CPU implementation leverages OpenMP work-sharing constructs to parallelize the search across the batch of queries and dataset sequences. To maximize parallel efficiency, each thread writes its local candidate minimum into a dedicated, thread-private result buffer, completely eliminating the need for mutual exclusion primitives such as `#pragma omp critical` or atomic updates. This architectural design decouples the execution paths, preventing thread stalls due to lock contention and strictly avoiding false sharing across cache lines. Furthermore, a `schedule(static)` directive is adopted: since all time series share an identical length and dimensional layout, the computational workload per iteration is perfectly uniform and deterministic. Under these conditions, static chunk partitioning eliminates runtime scheduling and synchronization overheads, as each worker thread deterministically receives an equal share of the problem space prior to loop execution.

```
#pragma omp parallel for schedule(static)
for (int s = 0; s < num_series; ++s) {
    const double* series_ptr = &dataset.all_data_flat[s * series_len];
    int best_idx = 0;
    double min_sad = std::numeric_limits<double>::max();

    for (int i = 0; i <= series_len - query_len; ++i) {
        double current_sad = 0.0;
        for (int j = 0; j < query_len; ++j) {
```

```

        current_sad += std::abs(series_ptr[i + j] - query[j]);
    }
    if (current_sad < min_sad) {
        min_sad = current_sad;
        best_idx = i;
    }
}
results[s] = best_idx;
}

```

Dim	100 (Queries: 5)			
Sequential	5.066 s			
Thread	Time AoS	Speedup AoS	Time SoA	Speedup SoA
2	2.682 s	1.89	2.714 s	1.87
4	1.560 s	3.25	1.575 s	3.22
8	1.115 s	4.54	1.092 s	4.64

Table 1: Speedup for $L = 100$ and $Q = 5$ varying the number of threads with OpenMP (best result in bold)

Dim	500 (Queries: 5)			
Sequential	26.181 s			
Thread	Time AoS	Speedup AoS	Time SoA	Speedup SoA
2	15.017 s	1.74	15.546 s	1.68
4	8.934 s	2.93	8.797 s	2.98
8	4.801 s	5.45	5.226 s	5.01

Table 2: Speedup for $L = 500$ and $Q = 5$ varying the number of threads with OpenMP

Dim	1000 (Queries: 5)			
Sequential	46.713 s			
Thread	Time AoS	Speedup AoS	Time SoA	Speedup SoA
2	26.680 s	1.75	26.507 s	1.76
4	15.625 s	2.99	15.555 s	3.00
8	8.538 s	5.47	8.603 s	5.43

Table 3: Speedup for $L = 1000$ and $Q = 5$ varying the number of threads with OpenMP

Dim	500 (Queries: 50)			
Sequential	250.084 s			
Thread	Time AoS	Speedup AoS	Time SoA	Speedup SoA
2	147.410 s	1.70	148.773 s	1.68
4	85.655 s	2.92	85.799 s	2.91
8	47.614 s	5.25	46.959 s	5.33

Table 4: Speedup for $L = 500$ and $Q = 50$ varying the number of threads with OpenMP

3.2 Analysis of OpenMP Performance and Scalability

The empirical results compiled across Tables 1–4 highlight several key architectural insights regarding multicore CPU scaling:

- **Multithreading Scalability:** The OpenMP parallelization achieves robust speedups across all tested workloads, scaling effectively from $1.70\times$ – $1.89\times$ with 2 threads up to a peak speedup of $5.47\times$ with 8 threads (Table 3). The scaling efficiency remains remarkably high up to 4 physical cores (parallel efficiency $\approx 73\%$ – 81%), validating the complete absence of lock contention and false sharing ensured by thread-private reduction buffers.
- **Simultaneous Multithreading (SMT) Effects:** Transitioning from 4 to 8 threads yields sub-linear scaling, with parallel efficiency decreasing to approximately 52% – 68% . This behavior stems from the hardware topology of the host processor, where hyper-threads share identical execution pipelines, ALUs, and L1/L2 cache capacity.
- **Data Layout Comparison (AoS vs. SoA):** Execution times between the AoS and SoA layouts on the CPU host remain closely aligned, with differences confined within 1% – 3% . Because the search evaluates univariate 1D time series, sequentially reading adjacent samples within each series exhibits nearly identical cache line spatial locality in both representations.
- **Workload Robustness:** Scaling the batch size by an order of magnitude (from $Q = 5$ to $Q = 50$) preserves the exact same speedup trends across all thread counts, demonstrating the deterministic load balancing achieved by the static work-sharing directive.

3.3 CUDA Kernel Design (GPU)

The CUDA acceleration strategy maps the multi-query sliding window search across a 2D grid hierarchy on the GPU:

- **Grid Topology:** Thread blocks are organized along a two-dimensional grid `dim3(num_series, num_queries)`, assigning one thread block to each (series, query) pair.
- **Memory Transfers:** The flattened SoA dataset is transferred from host memory to device VRAM once via `cudaMemcpy`, eliminating repetitive PCIe bus latency overheads across benchmark iterations.
- **Shared Memory Utilization:** At kernel launch, the target time-series is cooperatively staged into dynamically allocated Shared Memory. This design isolates query matching from high-latency global memory fetches and ensures that all threads within the block operate over contiguous, low-latency shared buffers.

- **Parallel Reduction Tree:** Local candidate SAD values are aggregated directly on-chip (Shared Memory) through a parallel binary tree reduction using `__syncthreads()` barriers, storing intermediate minimum distance-index pairs without serialized global atomics.

4 Experimental Protocol

1. **Cold-Start Discard (Warm-up):** A dedicated preliminary execution cycle ($W = 1$) is completed prior to benchmark measurements. This eliminates initial disk cache read penalties, operating system paging jitter, and the lazy initialization latency of the CUDA driver runtime context.
2. **Statistical Replications:** Each unique configuration of **query length** and **number of query** is repeated for $R = 5$ independent measurement runs. For each configuration, we compute the sample mean ($\bar{t}$), sample standard deviation (σ), minimum ($t_{\min}$), and maximum ($t_{\max}$):

$$\bar{t} = \frac{1}{R} \sum_{r=1}^R t_r, \quad \sigma = \sqrt{\frac{1}{R-1} \sum_{r=1}^R (t_r - \bar{t})^2} \quad (4)$$

3. **Cross-Validation of Correctness:** Results generated by the GPU are verified against the CPU reference baseline. The validation handles floating-point ties by identifying equivalent global minimum:

$$|\mathcal{D}_{\text{GPU}} - \mathcal{D}_{\text{CPU}}| \leq 10^{-4} \quad (5)$$

accounting for differing index selections during tree reductions across parallel warps.

4.1 Benchmarking Hardware Specifications

- **Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz:** Multicore x86-64 host processor configured to evaluate thread scalability ($P \in \{1, 2, 4, 8\}$).
- **Cloud GPU Platform (Google Colab):** Dual-core virtualized host paired with an **NVIDIA Tesla T4 GPU** (Turing architecture, 40 Streaming Multiprocessors, 2560 CUDA cores, 16GB GDDR6 VRAM, compute capability 7.5).

5 Experimental Results

The benchmark evaluates the *FaultDetectionA* dataset, tracking timing variations across query lengths $L \in \{100, 500, 1000\}$ and query batch sizes $Q \in \{5, 50\}$.

5.1 Empirical Tuning of Thread Block Granularity

To optimize occupancy and determine the optimal thread block dimension, an empirical tuning phase was conducted under a fixed representative workload ($L = 500, Q = 50$). Candidate block dimensions $B \in \{32, 64, 128, 256, 512\}$ were evaluated.

Table 5: Empirical tuning of CUDA thread block size ($L = 500$, $Q = 50$, $R = 5$ runs).

Block Size (B)	Mean Time [ms]	Std. Dev. [ms]	Min Time [ms]	Relative Speedup
32	9996.72	± 32.553	9962.14	$1.00\times$ (Baseline)
64	6672.28	± 19.828	6655.63	$1.50\times$
128	5995.49	± 0.065	5995.40	$1.67\times$
256	5971.59	± 0.047	5971.54	$1.67\times$
512	5961.47	± 0.029	5961.44	$1.68\times$

At $B = 32$, the single-warp configuration suffers from severe latency under-utilization. Scaling to $B = 128$ yields a $1.67\times$ speedup, after which performance plateaus. Crucially, attempting $B = 1024$ triggers an invalid configuration error: with a series length of 5120 double elements (≈ 40.96 KB) plus reduction buffers, $B = 1024$ requires 56 KB of shared memory, exceeding the strict 48 KB hardware limit per thread block on the Turing architecture. Therefore, $B = 128$ represents the most balanced configuration, achieving peak throughput while minimizing shared memory footprint.

5.2 CUDA Performance on NVIDIA Tesla T4

Table 6 presents the empirical measurements gathered from the CUDA execution runs using the tuned block configuration.

Table 6: CUDA performance on NVIDIA Tesla T4 using Structure of Arrays (SoA) layout ($R = 5$ runs).

Query Length (L)	Num Queries (Q)	Mean Time [ms]	Std. Dev. [ms]	Min Time [ms]	Max Time [ms]
100	5	135.092	± 0.253	134.866	135.387
100	50	1341.350	± 2.188	1337.950	1343.180
500	5	598.047	± 0.265	597.902	598.520
500	50	5976.920	± 14.019	5961.250	5988.680
1000	5	1059.170	± 2.573	1057.330	1063.330
1000	50	10783.900	± 170.446	10614.400	10992.200

The recorded standard deviations remain within 0.1%–1.5% of the sample means across all batch sizes, demonstrating consistent kernel execution stability. Scaling from $Q = 5$ to $Q = 50$ scales total execution time almost exactly by a linear factor of $10\times$, proving excellent throughput scalability.

5.3 Speedup and Efficiency Evaluation

Speedup (S) is evaluated against the single-threaded CPU baseline (T_{seq}):

$$S = \frac{T_{\text{seq}}}{T_{\text{parallel}}} \quad (6)$$

Table 7 presents the cross-platform comparative synthesis across all evaluated configurations.

Table 7: Global performance comparison between Sequential CPU, OpenMP (8 threads), and CUDA.

Length (L)	Batch (Q)	Sequential CPU	OpenMP AoS (8T)	OpenMP SoA (8T)	CUDA SoA	CUDA Speedup
100	5	5.07 s	1.12 s	1.09 s	135.09 ms	37.50 $\times$
100	50	45.47 s	10.99 s	11.00 s	1341.35 ms	33.90 $\times$
500	5	26.18 s	4.80 s	5.23 s	598.05 ms	43.78 $\times$
500	50	250.08 s	47.61 s	46.96 s	5976.92 ms	41.84 $\times$
1000	5	46.71 s	8.54 s	8.60 s	1059.17 ms	44.10 $\times$
1000	50	451.40 s	83.37 s	83.11 s	10783.90 ms	41.86 $\times$

6 Discussion and Conclusions

The comparative findings highlight critical systems-level trade-offs:

1. **Impact of Data Layout:** Transitioning from AoS to SoA prevents cache fragmentation on the CPU host and unlocks contiguous memory transactions on the GPU, yielding notable data staging efficiency.
2. **CPU Multithreading via OpenMP:** OpenMP provides near-linear scaling up to 4 cores and achieves speedups up to 5.47 $\times$ on 8 logical threads using static scheduling and thread-private buffers. However, execution remains bounded by memory bandwidth during lengthy scans.
3. **Massive GPU Acceleration:** Offloading the search to the NVIDIA Tesla T4 delivers peak speedups between **33.9** $\times$ and **44.1** $\times$ over the sequential CPU baseline. Shared memory staging and parallel tree reduction effectively overcome global memory latency.

References

- [1] R. Strzodka, “Abstraction for AoS and SoA Layout in C++,” in *GPU Computing Gems Jade Edition*, W. W. Hwu, Ed. Morgan Kaufmann, 2012, pp. 429–441.
- [2] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: The MIT Press, 2007.
- [3] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable Parallel Programming with CUDA,” *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008.
- [4] B. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, “The UEA & UCR Time Series Classification Repository,” *arXiv preprint arXiv:1602.01711*, 2017. [Online]. Available: <https://www.timeseriesclassification.com/description.php?Dataset=FaultDetectionA>